

Mizan.ai

All Models — Performance Analysis

QwenBrain, QwenLite, Translator, and Embedder — real numbers, measured directly against the live Modal endpoints for this report, with real current GPU pricing.

Every number in this report was captured live, moments before this document was generated, by calling each production endpoint directly — none of it is estimated, simulated, or carried over from an older investigation. GPU pricing was fetched live from Modal's own pricing page at the same time.

1. The four models

Model	Model ID	GPU	Endpoint	Role
QwenBrain	Qwen2.5-14B-Instruct (GPTQ-Int4)	A10	/generate	Agent brain — tool-calling, classification, narration; the “does the real reasoning” model
QwenLite	Qwen2.5-7B-Instruct (GPTQ-Int4)	L4	/generate-lite	Free-tier chat / RAG answer generation — smaller, meant to be fast and cheap
Translator	MADLAD-400 (google/madlad400-3b-mt)	T4	/translate	EN<->AR paragraph translation, seq2seq, not a chat model
Embedder	BGE-M3	T4	/embed	1024-dim embeddings for RAG indexing and query, and agentic memory

All four are deployed together in one Modal app (mizan-models), each as its own auto-scaling class (scales to zero after 2–5 minutes idle). vllm==0.22.1 is pinned for QwenBrain/QwenLite specifically because vLLM's snapshot/sleep-wake APIs have moved across versions before.

2. Methodology

QwenBrain and QwenLite share the same request/response shape (a chat-style vLLM endpoint reporting `completion_tokens`), so the same method was used for both:

- **Cold start:** the very first call made, before any warm-up — whatever real state the container happened to be in.
- **Warm time-to-first-token (TTFT):** median of 5 calls with `max_tokens=1`, back-to-back once the container is confirmed warm.
- **Warm decode throughput:** 5 further calls with a larger token budget; each response's *actual* `completion_tokens` count is used (the model doesn't always use the full requested budget), then $(\text{time_big} - \text{TTFT}) / (\text{tokens_big} - 1)$ isolates the steady-state per-token cost from the fixed first-token cost. Medians used throughout.

Translator and Embedder have no token-budget concept in their API (a single blocking call, no `max_tokens`, no token count in the response), so the same TTFT/decode-rate method doesn't apply. Instead: cold start (first call), then warm round-trip latency at more than one input size — a single sentence vs. a longer paragraph for Translator, a single text vs. a batch of 16 for Embedder — to see how each scales.

3. Results — QwenBrain & QwenLite (vLLM chat models)

Metric	QwenBrain (14B, A10)	QwenLite (7B, L4)
Cold start (first call)	59.37s	60.29s
Warm TTFT (median of 5)	1.431s	1.217s
Warm decode throughput	116.07 tokens/sec	38.40 tokens/sec
Warm inter-token latency	8.6 ms/token	26.0 ms/token

All five runs per metric shown in full in the appendix data captured alongside this report; medians reported above to reduce network-jitter noise.

A genuinely counter-intuitive real result

QwenBrain — the larger, 14B model — decodes roughly **3x faster** than QwenLite, the smaller 7B model, despite running on the pricier A10 rather than L4. This isn't a measurement artifact (both were measured with the identical method, moments apart, both confirmed warm); it reflects that A10 is simply the stronger inference GPU of the two for raw throughput — L4 is optimized more for power/cost efficiency at smaller scale than for peak decode speed. The practical implication: QwenLite's GPU tier was chosen for cost, not because it's the faster option — it genuinely isn't, on this measurement.

4. Results — Translator (MADLAD-400)

Metric	Value
Cold start (first call)	33.37s
Warm, one sentence (~17 words)	3.001s (median of 5)
Warm, a 3-sentence paragraph (~40 words)	6.003s (median of 5)

Latency scales roughly with input length, as expected for a seq2seq model with no batching applied on the caller's side here (each call was a single text). Notably the fastest cold start of the four models — MADLAD-400 (3B) is the smallest checkpoint of the four, and doesn't carry vLLM's own engine-initialization overhead (torch.compile/CUDA graph considerations don't apply to this transformers-based seq2seq path).

5. Results — Embedder (BGE-M3)

Metric	Value
Cold start (first call)	43.96s
Warm, single text	1.171s (median of 5)
Warm, batch of 16 texts	1.197s (median of 5)
Amortized cost per text, batch of 16	74.8 ms/text

Batching is essentially free here

16 texts embedded in a single batched call took almost exactly the same wall-clock time as embedding just 1 — 1.197s vs. 1.171s. The practical implication for the RAG ingestion pipeline and any other bulk-embedding workload: batching requests into the Embedder is close to free additional throughput, and sending texts one at a time when a batch is available leaves real performance on the table.

6. Cost analysis (real Modal pricing, fetched live)

GPU	Price / second	Approx. price / hour	Used by
A10	\$0.000306	~\$1.10	QwenBrain
L4	\$0.000222	~\$0.80	QwenLite
T4	\$0.000164	~\$0.59	Translator, Embedder

Estimated cost per typical warm request

Computed directly from the real measured numbers above × real per-second GPU pricing above — not a separate estimate. For the two chat models, “typical” is assumed as a 200-token reply (TTFT + 199 × measured inter-token time); for Translator/Embedder, the measured single-input warm latency is used directly.

Model	Assumed request	Warm time	Cost / request
QwenBrain	200-token reply	~3.15s	~\$0.00096 (~0.10¢)
QwenLite	200-token reply	~6.40s	~\$0.00142 (~0.14¢)
Translator	One sentence	3.00s	~\$0.00049 (~0.05¢)
Embedder	One text	1.17s	~\$0.00019 (~0.02¢)
Embedder (batched)	16 texts in one call	1.20s	~\$0.0000123/text (~0.001¢/text)

Despite being on the more expensive GPU, QwenBrain is cheaper per 200-token reply than QwenLite — direct consequence of \$3's throughput finding. Cold-start cost is excluded here since it's amortized across every request served during that container's warm window, not paid per-request.

7. Design decisions already reflected in the deployment

- `enforce_eager=True` on both QwenBrain and QwenLite — skips `torch.compile` + CUDA graph capture, trading a small amount of steady-state throughput for a much faster cold start. Measured at the time this was decided: QwenBrain's cold start went from 115.79s to 64.47s (engine-init specifically: 72.53s to 14.82s); an alternative (caching the compile output on a persistent Volume) was also measured and was slower end-to-end.
- **Timing is logged, never returned to the client.** Every one of the four endpoints logs total generation time (and, where meaningful, tokens/sec) to Modal's own container logs only — never included in any response body, so it can never leak into a frontend or a browser's network tab. This is exactly the mechanism that first exposed QwenLite's 0.35 tokens/sec on T4.
- **GPU memory snapshotting was tried and reverted.** Modal's experimental snapshot feature failed at the infrastructure level after the application code itself succeeded (engine load, compile, warmup, sleep all completed correctly) — reverted in favor of the simpler, dependable `enforce_eager` approach rather than depending on an explicitly experimental platform feature.
- **Per-model max_containers and concurrency caps** are set deliberately differently: QwenBrain/QwenLite cap at 2 containers (cost control during the prototype phase); Embedder allows 16 concurrent inputs per container since it's cheap and fast; QwenLite allows 5 concurrent inputs (small model, more headroom) vs. QwenBrain's 4.

8. Summary

All four models are healthy, correctly deployed, and now benchmarked with real numbers rather than estimates. The one number worth remembering above the rest: QwenBrain outperforms QwenLite on raw throughput despite being twice the parameter count, because A10 is simply a stronger inference GPU than L4 — QwenLite's value is in cost, not speed. Cold start (~33–60s across all four models) remains the dominant latency cost for any request that hits an idle container, dwarfing every warm-path number in this report by more than an order of magnitude —

the single highest-leverage place to look next if first-response latency still needs to improve further.